E-Commerce Customer Churn Analysis

Project Steps and Objectives:

Data Cleaning:

Handling Missing Values and Outliers:

```
SET SQL_SAFE_UPDATES = 0;
```

➢ Impute mean for the following columns, and round off to the nearest integer if required: WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear, DaySinceLastOrder.

```
UPDATE customer_churn
SET WarehouseToHome = (SELECT wh FROM
(SELECT ROUND (AVG(WarehouseToHome)) AS wh FROM customer_churn) cc)
WHERE WarehouseToHome IS NULL;

UPDATE customer_churn
SET HourSpendOnApp = (SELECT hs FROM
(SELECT ROUND (AVG(HourSpendOnApp)) AS hs FROM customer_churn) cc)
WHERE HourSpendOnApp IS NULL;

UPDATE customer_churn
SET OrderAmountHikeFromlastYear = (SELECT oa FROM
(SELECT ROUND (AVG(OrderAmountHikeFromlastYear)) AS oa FROM
customer_churn) cc) WHERE OrderAmountHikeFromlastYear IS NULL;
```

```
UPDATE customer_churn
SET DaySinceLastOrder = (SELECT ds FROM
(SELECT ROUND (AVG(DaySinceLastOrder)) AS ds FROM customer_churn) cc)
WHERE DaySinceLastOrder IS NULL;
```

➢ Impute mode for the following columns: Tenure, CouponUsed, OrderCount.

```
UPDATE customer_churn
SET Tenure = (SELECT t FROM (SELECT Tenure AS t FROM customer_churn
WHERE Tenure IS NOT NULL GROUP BY Tenure ORDER BY COUNT(*) DESC LIMIT 1) cc)
WHERE Tenure IS NULL;
```

```
UPDATE customer_churn
SET CouponUsed = (SELECT c FROM (SELECT CouponUsed AS c FROM
customer_churn  WHERE CouponUsed IS NOT NULL GROUP BY CouponUsed
ORDER BY COUNT(*) DESC LIMIT 1) cc) WHERE CouponUsed IS NULL;
```

```
UPDATE customer_churn
SET OrderCount = (SELECT o FROM (SELECT OrderCount AS o FROM customer_churn
WHERE OrderCount IS NOT NULL GROUP BY OrderCount ORDER BY COUNT(*) DESC
LIMIT 1) cc) WHERE OrderCount IS NULL;
```

➢ Handle outliers in the 'WarehouseToHome' column by deleting rows where the values are greater than 100.

```
DELETE FROM customer_churn WHERE WarehouseToHome > 100;
```

Dealing with Inconsistencies:

➢ Replace occurrences of “Phone” in the 'PreferredLoginDevice' column and “Mobile” in the 'PreferedOrderCat' column with “Mobile Phone” to ensure uniformity.

```
UPDATE customer_churn
SET PreferredLoginDevice = 'Mobile Phone'
WHERE PreferredLoginDevice = 'Phone';

UPDATE customer_churn
SET PreferedOrderCat = 'Mobile Phone'
WHERE PreferedOrderCat = 'Mobile';
```

➢ Standardize payment mode values: Replace "COD" with "Cash on Delivery" and "CC" with "Credit Card" in the PreferredPaymentMode column.

```
UPDATE customer_churn
SET PreferredPaymentMode = 'Cash on Delivery'
WHERE PreferredPaymentMode = 'COD';

UPDATE customer_churn
SET PreferredPaymentMode = 'Credit Card'
WHERE PreferredPaymentMode = 'CC';
```

Data Transformation:

Column Renaming:

➢ Rename the column "PreferedOrderCat" to "PreferredOrderCat".

```
ALTER TABLE customer_churn
CHANGE PreferedOrderCat PreferredOrderCat  VARCHAR(20);
```

➢ Rename the column "HourSpendOnApp" to "HoursSpentOnApp".

```
ALTER TABLE customer_churn
CHANGE HourSpendOnApp HoursSpentOnApp INT;
```

Creating New Columns:

➢ Create a new column named ‘ComplaintReceived’ with values "Yes" if the corresponding value in the ‘Complain’ is 1, and "No" otherwise.

```
ALTER TABLE customer_churn ADD COLUMN ComplaintReceived VARCHAR(3);
UPDATE customer_churn
SET ComplaintReceived = IF(Complain = 1, 'Yes', 'No');
```

➢ Create a new column named 'ChurnStatus'. Set its value to “Churned” if the corresponding value in the 'Churn' column is 1, else assign “Active”.

```
ALTER TABLE customer_churn ADD COLUMN ChurnStatus VARCHAR(10);
UPDATE customer_churn
SET ChurnStatus = IF(Churn = 1, 'Churned', 'Active');
```

Column Dropping:

➢ Drop the columns "Churn" and "Complain" from the table.

```
ALTER TABLE customer_churn
DROP COLUMN Churn,
DROP COLUMN Complain;
```

Data Exploration and Analysis:

➢ Retrieve the count of churned and active customers from the dataset.

```
SELECT ChurnStatus, COUNT(*) AS CustomerCount
FROM customer_churn
GROUP BY ChurnStatus;
```

➢ Display the average tenure and total cashback amount of customers who churned.

```
SELECT ROUND(AVG(Tenure), 2) AS AvgTenure,
SUM(CashbackAmount) AS TotalCashback FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

➢ Determine the percentage of churned customers who complained.

```
SELECT ROUND(100 * SUM(ComplaintReceived = 'Yes') / COUNT(*), 2) AS
PctChurnedComplained FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

➢ Identify the city tier with the highest number of churned customers whose preferred order category is Laptop & Accessory.

```
SELECT CityTier, COUNT(*) AS ChurnedCustomers FROM customer_churn
WHERE ChurnStatus = 'Churned' AND PreferredOrderCat = 'Laptop & Accessory'
GROUP BY CityTier
ORDER BY ChurnedCustomers DESC
LIMIT 1;
```

➢ Identify the most preferred payment mode among active customers.

```
SELECT PreferredPaymentMode, COUNT(*) AS Customers
FROM customer_churn
WHERE ChurnStatus = 'Active'
GROUP BY PreferredPaymentMode
ORDER BY Customers DESC
LIMIT 1;
```

➢ Calculate the total order amount hike from last year for customers who are single and prefer mobile phones for ordering.

```
SELECT SUM(OrderAmountHikeFromlastYear) AS TotalOrderAmountHike
FROM customer_churn
WHERE MaritalStatus = 'Single' AND PreferredOrderCat = 'Mobile Phone';
```

➢ Find the average number of devices registered among customers who used UPI as their preferred payment mode.

```
SELECT ROUND(AVG(NumberOfDeviceRegistered), 2) AS AvgDevices
FROM customer_churn
WHERE PreferredPaymentMode = 'UPI';
```

➢ Determine the city tier with the highest number of customers.

```
SELECT CityTier, COUNT(*) AS Customers FROM customer_churn
GROUP BY CityTier
ORDER BY Customers DESC
LIMIT 1;
```

➢ Identify the gender that utilized the highest number of coupons.

```
SELECT Gender, SUM(CouponUsed) AS TotalCoupons FROM customer_churn
GROUP BY Gender
ORDER BY TotalCoupons DESC
LIMIT 1;
```

➢ List the number of customers and the maximum hours spent on the app in each preferred order category.

```
SELECT PreferredOrderCat, COUNT(*) AS Customers,
MAX(HoursSpentOnApp) AS MaxHoursOnApp FROM customer_churn
GROUP BY PreferredOrderCat;
```

➢ Calculate the total order count for customers who prefer using credit cards and have the maximum satisfaction score.

```
SELECT SUM(OrderCount) AS TotalOrderCount
FROM customer_churn
WHERE PreferredPaymentMode = 'Credit Card'
AND SatisfactionScore = (SELECT MAX(SatisfactionScore) FROM customer_churn);
```

➢ What is the average satisfaction score of customers who have complained?

```
SELECT ROUND(AVG(SatisfactionScore), 2) AS AvgSatisfaction
FROM customer_churn
WHERE ComplaintReceived = 'Yes';
```

➢ List the preferred order category among customers who used more than 5 coupons.

```
SELECT PreferredOrderCat, COUNT(*) AS Customers FROM customer_churn
WHERE CouponUsed > 5
GROUP BY PreferredOrderCat
ORDER BY Customers DESC;
```

➢ List the top 3 preferred order categories with the highest average cashback amount.

```
SELECT PreferredOrderCat, ROUND(AVG(CashbackAmount), 2) AS AvgCashback
FROM customer_churn
```

```
GROUP BY PreferredOrderCat
ORDER BY AvgCashback DESC
LIMIT 3;
```

➢ Find the preferred payment modes of customers whose average tenure is 10 months and have placed more than 500 orders.

```
SELECT PreferredPaymentMode FROM customer_churn
GROUP BY PreferredPaymentMode
HAVING AVG(Tenure) = 10 AND SUM(OrderCount) > 500;
```

➢ Categorize customers based on their distance from the warehouse to home such as 'Very Close Distance' for distances <=5km, 'Close Distance' for <=10km, 'Moderate Distance' for <=15km, and 'Far Distance' for >15km. Then, display the churn status breakdown for each distance category.

```
SELECT CASE WHEN WarehouseToHome <= 5  THEN 'Very Close Distance'
            WHEN WarehouseToHome <= 10 THEN 'Close Distance'
            WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'
            ELSE 'Far Distance' END AS DistanceCategory,
       ChurnStatus,
       COUNT(*) AS Customers
FROM customer_churn
GROUP BY DistanceCategory, ChurnStatus;
```

➢ List the customer’s order details who are married, live in City Tier-1, and their order counts are more than the average number of orders placed by all customers.

```
SELECT * FROM customer_churn
WHERE MaritalStatus = 'Married'
  AND CityTier = 1
  AND OrderCount > (SELECT AVG(OrderCount) FROM customer_churn);
```

➢ a) Create a ‘customer_returns’ table in the ‘ecomm’ database and insert the following data:

ReturnID	CustomerID	ReturnDate	RefundAmount
1001	50022	2023-01-01	2130
1002	50316	2023-01-23	2000
1003	51099	2023-02-14	2290
1004	52321	2023-03-08	2510
1005	52928	2023-03-20	3000
1006	53749	2023-04-17	1740
1007	54206	2023-04-21	3250
1008	54838	2023-04-30	1990

```
CREATE TABLE customer_returns (
   ReturnID     INT PRIMARY KEY,
   CustomerID   INT,
   ReturnDate   DATE,
   RefundAmount INT
);

INSERT INTO customer_returns VALUES
(1001, 50022, '2023-01-01', 2130),
(1002, 50316, '2023-01-23', 2000),
(1003, 51099, '2023-02-14', 2290),
(1004, 52321, '2023-03-08', 2510),
```

(1005, 52928, '2023-03-20', 3000),

(1006, 53749, '2023-04-17', 1740),

(1007, 54206, '2023-04-21', 3250),

(1008, 54838, '2023-04-30', 1990);

b) Display the return details along with the customer details of those who have churned and have made complaints.

```
SELECT r.ReturnID, r.ReturnDate, r.RefundAmount, c.*
FROM customer_returns r
JOIN customer_churn c ON c.CustomerID = r.CustomerID
WHERE c.ChurnStatus = 'Churned' AND c.ComplaintReceived = 'Yes';
```